JavaScript DOM Manipulation

Deep Dive — Lesson Notes

Grade 4 | Beginner-Friendly | Illustrated Examples

✥ Understanding the Document Object Model ✥

Lesson Overview

■ **Subject**	Computer Science / Web Development
■ **Grade**	Grade 4 (Ages 9–10)
■ **Duration**	45–60 minutes
■ **Pages**	Detailed notes with code examples & activities
■ **Context**	Examples from everyday Indian life (cricket, rangoli, chai, WhatsApp)

Table of Contents

1. What is the DOM?

Document Object Model — Your Webpage's Blueprint

When a web browser loads an HTML page it creates an internal map of every element on the page. This map is called the **Document Object Model (DOM)**. Think of it as a living, interactive blueprint — JavaScript can read it, change it, add to it, or remove parts from it, all without ever reloading the page.

Real-Life Analogy	How It Works	DOM Equivalent
Diwali Rangoli	A pattern made of coloured powder — you can change any colour anytime.	DOM elements whose style can be updated instantly.
Cricket Scorecard	A board that shows runs. When a boundary is hit, only the score box updates.	Changing one element's content without touching others.
School Notice Board	Paper slips pinned on a board. A teacher can replace one slip with a new message.	Adding or replacing child elements inside a parent.

■ Demo Idea

Show students a simple web page in a browser. Open DevTools (F12) → Elements tab. Click on a heading and edit the text live. This is DOM manipulation in action!

2. The DOM Tree

A Family Tree for Your Webpage

Every HTML document is structured like a family tree. The outermost element is the **ancestor**, and it has children, grandchildren, and so on. This nested structure is called the **DOM Tree**.

Visualising the Tree:

```
html ← 'Dadi' — the root of everything
■■■ head
■ ■■■ title
■■■ body
■■■ h1 ← first child of body
■■■ p ← second child
■■■ p ← third child
■■■ button ← fourth child
```

Key Vocabulary

Term	Example Tag	Meaning
Root Element	<html>	The topmost ancestor — like Dadi in a joint family.
Parent Element	e.g. <body>	An element that directly contains other elements.
Child Element	e.g. <h1>, <p>	An element directly inside a parent.
Sibling Elements	e.g. <h1> & <p>	Elements that share the same parent.

■ Discussion Prompt

Ask students: 'In your family who is the Dadi? Who are the children? Who are the siblings?' Then map those roles onto the HTML tags on the board.

3. JavaScript — The Helper

The Tool That Brings Your Webpage to Life

HTML gives a page its **structure** (the bones). CSS gives it **style** (the clothes). JavaScript gives it **behaviour** (the ability to do things). When we talk about DOM Manipulation, we are using JavaScript to find elements in the DOM tree and change them.

What Can JavaScript Do to the DOM?

Find parts of the page	Like calling a student's roll number — the element stands up.
Change text or content	Like updating a WhatsApp status — instantly visible to everyone.
React to user actions	Like a doorbell — press it and something happens!
Add / Remove elements	Like adding a new notice to the school board, or taking one down.

4. Selecting Elements: getElementById()

Finding ONE specific element by its unique ID

Before JavaScript can change anything on a page it must first **find** the element. The most common way for beginners is **getElementById()**. It searches the entire DOM tree and returns the element whose id attribute matches exactly.

01

Why IDs are Unique

Just like every Indian citizen has a unique Aadhaar number, every HTML element with an id must have a different value. No two elements can share the same id on one page. This guarantees getElementById() finds exactly ONE element.

02

School Roll Number Analogy

When your teacher calls Roll Number 17, only one student stands up. getElementById('myHeading') works the same way — only one element responds.

Syntax

```
// HTML side:
Hello Duniya!

// JavaScript side:
let heading = document.getElementById('myHeading');
// 'heading' now points to the element above.
// We can now read or change it!
```

■ Common Mistakes to Avoid

- Spelling: getElementById — the 'B' in 'By' is capital, the 'I' in 'Id' is capital, everything else lowercase.
- Quotes: The id must be inside quote marks — single or double.
- Typos in the id: 'myHeading' in JavaScript must match id="myHeading" in HTML exactly — case-sensitive!

5. Changing Content: .innerHTML

Updating what is written inside any element

Once you have selected an element you can read or write its content using the **.innerHTML** property. Think of it like the menu board at your local chai shop — the shopkeeper can erase the old price and write a new one instantly.

Reading innerHTML

```
let heading = document.getElementById('myHeading');
console.log(heading.innerHTML); // prints: Hello Duniya!
```

Writing innerHTML (changing text)

```
let heading = document.getElementById('myHeading');
heading.innerHTML = 'Namaste, India!';
// The heading on screen immediately changes to 'Namaste, India!'
```

innerHTML can also include HTML tags

```
let box = document.getElementById('myBox');
box.innerHTML = '<b>Bold text</b> and <i>italic text</i>';
// The browser renders the tags — text appears bold and italic
```

innerHTML vs textContent

Property	What it Does	Example
.innerHTML	Gets/sets HTML content (including tags).	box.innerHTML = 'Hi'
.textContent	Gets/sets plain text only — ignores any HTML tags.	box.textContent = 'Hi'

6. onclick Events

Making things happen when the user clicks

An **event** in JavaScript is something that happens — a click, a keypress, moving the mouse. The **onclick** attribute (or event listener) lets us run a JavaScript function whenever a button (or any element) is clicked. Think of clicking as ringing a temple bell — the moment you press it, an action is triggered!

Method 1 — inline onclick attribute (simplest)

```
Badlo! (Change!)

// JavaScript
function changeHeading() {
document.getElementById('myHeading').innerHTML = 'Namaste, India!';
}
```

Method 2 — addEventListener (best practice)

```
Badlo!

// JavaScript
document.getElementById('myBtn').addEventListener('click',
changeHeading);

function changeHeading() {
document.getElementById('myHeading').innerHTML = 'Namaste, India!';
}
```

■ Differentiation Note

For Grade 4 students the inline onclick method is perfectly fine. Mention addEventListener as a preview of what they will learn next year.

7. Writing Functions in the <script> Tag

Where does JavaScript code live?

JavaScript code must be wrapped inside a **<script>** tag. Think of the HTML file as a recipe card: the ingredients list is your HTML, and the cooking steps are your JavaScript — both live on the same card.

Complete Working Example

```
Hello Duniya!
Badlo! (Change!)

<br/> function changeHeading() {<br/>
document.getElementById('myHeading').innerHTML = 'Namaste,
India!';<br/> }<br/>
```

■ Key Rules for <script>

- Place <script> just before </body> so HTML loads first.
- Every function uses curly braces { } to wrap its steps.
- Each instruction ends with a semicolon ; (good habit!).
- JavaScript is case-sensitive: getElementById ≠ getelementbyid.

8. Full Flow Recap

Putting it all together — step by step

Let's trace exactly what happens from the moment a user clicks a button to the moment the page text changes:

Step 1

User clicks the button

The browser detects the click event on the <button> element.

Step 2

onclick fires the function

The function name written in onclick="..." is called automatically.

Step 3

getElementById() finds the element

JavaScript searches the DOM tree for the element with the matching id.

Step 4

.innerHTML changes the content

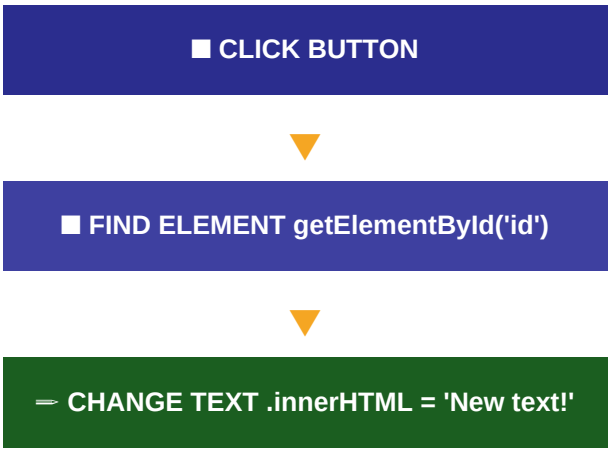
The text inside the found element is replaced with the new string.

Step 5

Browser re-renders

The browser sees the change and updates what is shown on screen — instantly!

Visual Flow Diagram



9. Check-in Quiz

Test Your Understanding!

Answer these questions. Answers are provided for the teacher — cover them during class!

Q1. What is the DOM like?

a) Rangoli b) TV c) Car

■ **Answer: a) Rangoli — because both have parts that can be repainted/changed.**

Q2. What does getElementById() do?

a) Find one element b) Change colour c) Play music

■ **Answer: a) Find one element — it returns the element with the given unique id.**

Q3. What property do we use to change the text inside an element?

a) .value b) .innerHTML c) .src

■ **Answer: b) .innerHTML — it lets us read and write the HTML content of an element.**

Q4. What happens when a user clicks a button with onclick?

a) The page reloads b) Nothing c) The linked function is called

■ **Answer: c) The linked function is called — just like pressing a doorbell makes it ring!**

Q5. Where does JavaScript code live in an HTML file?

a) Inside <style> b) Inside <script> c) Inside <head> only

■ **Answer: b) Inside <script> tags — placed just before </body> is best practice.**

10. Hands-On Activity

My Magic Name Board

Students will create a simple webpage that displays a heading and changes it to their own name when a button is clicked. This puts together everything learned in the lesson!

Complete Code for Students to Type

```
My Magic Name Board



Virat Kohli
Show My Name!

<br/> function changeName() {<br/> // Replace 'Arjun' with YOUR
name!<br/> document.getElementById('nameBoard').innerHTML =
'Arjun';<br/> }<br/>
```

Step-by-Step Instructions

1. Open a text editor (Notepad on Windows / TextEdit on Mac).
2. Type the code above exactly as shown.
3. Change **'Arjun'** to your own name inside the function.
4. Save the file as **nameboard.html** on your Desktop.
5. Open the file in Google Chrome by double-clicking it.
6. Click the button and watch your name appear!
7. **Extension:** Can you change the button text to say something in Hindi?

■ Teacher Tips

Walk around the room while students type. Common errors: missing quotes, wrong capitalisation in getElementById, and forgetting the semicolon after the innerHTML assignment. Celebrate each student the moment their name appears!

11. Key Takeaways & Quick Reference

Everything we learned today — at a glance

■ DOM	The Document Object Model is the browser's internal map of all HTML elements. JavaScript can read and change this map.
■ getElementById()	Finds exactly ONE element using its unique id — like calling a roll number in class.
✏ .innerHTML	Reads or writes the HTML content inside an element — like updating a chai shop menu board.
■ onclick	Runs a JavaScript function when an element is clicked — like ringing a doorbell.
■ <script>	JavaScript code lives inside <script> tags, usually placed just before </body>.

Quick Reference Card

```
// 1. Select an element
let el = document.getElementById('myId');

// 2. Read its content
console.log(el.innerHTML);

// 3. Change its content
el.innerHTML = 'New text here!';

// 4. Attach a click handler
el.onclick = function() {
el.innerHTML = 'You clicked me!';
};

// 5. Or use inline onclick in HTML:
// Click me
```

12. Homework & What's Next

Keep the magic going at home!

Tonight's Homework

1. Open your **nameboard.html** file from today's activity.
2. Add a **second button** that changes the heading back to the original text (e.g. 'Virat Kohli').
3. Try changing the button label to a Hindi word that means 'change'.
4. **Bonus:** Show the magic to a family member and explain what getElementById() does — teach it to someone else!

Coming Up in the Next Lesson

■ **Changing Styles with .style** — Change colours, font sizes, and backgrounds using JavaScript.

■■■■■ **querySelectorAll()** — Select multiple elements at once using CSS selectors.

■ **Adding & Removing Elements** — Create new elements with createElement() and appendChild().

■ **Timers with setTimeout()** — Make things happen automatically after a delay.

■ Remember: Every great web developer started exactly where you are now. The magic of JavaScript is that a few simple lines of code can make a webpage come alive. Keep practising — you are already a coder! ■